Advanced C, part 2 of 3

INDEX Previous Next

Comprehensive listing of interrupts and hardware details.

CONTENTS OF PART TWO

VIDEO SCREEN DISPLAYS

Monochrome The monochrome screen is arranged as 80 characters (columns) by 25 lines (rows). The starting address in memory where the display buffer (4kbytes) starts is B000:0000. There is only one display page, and each character is stored followed by its attribute in the next byte, eg,

```
Address         Row,Column
B000:0000       Character 0,0
B000:0001       Attribute 0,0
B000:0002       Character 0,1
B000:0003       Attribute 0,1
```

The attribute byte is made up as follows,

```
Bit
7, BL = Blink
6 5 4, Background
3, I = Intensity or Highlight
2 1 0, Foreground
Back  Foregnd
 000   000   None
 000   001   Underline
 000   111   Normal video
 111   000   Reverse video
```

The offset of a character at a given row, column position is specified by the formula,

```
offset = (( row * 0x50 + column ) * 2 )
```

Color Graphics Adapter

This adapter supports the following display modes, using a 16kbyte buffer starting at B800:0000

```
Text modes 40.25 BW/Color 80.25 BW/Color
Graphic modes 320.200 BW/4 Color 640.200 BW
```

In text mode, the adapter uses more than one display page, but only one page is active at any one time. The user may set any page to be the active one. The fourty column modes support 8 display pages (0-7), whilst the eighty column modes support 4 display pages (0-3).

You may write to a page which is not currently active, then use the function call setpage() to switch to that page. Using direct memory addressing on the active page will result in snow (white lines) due to memory contention problems between the display controller and the central processor trying to use the display RAM at the same time.

The formula to derive the memory offset of a character at a given row/column position is,

```
offset = (( row * 0x50 + column ) * 2 ) + ( pagenum * 0x1000 )
```

The following program illustrates some of these concepts,

```
#include <dos.h> /* TITLE 80.25 Color adapter */
#include <stdio.h>
union REGS regs;

void setpage( unsigned int pagenum ) {
        regs.h.ah = 5; regs.h.al = pagenum; int86( 0x10, &regs, &regs );
}

main() {
        int x, y, offset;
        char attr, ch;
        char far *scrn = ( char far * ) 0xB8000000;
        char *message = "This was written direct to page zero whilst page
        /* set video mode to 80.25 color */
        regs.h.ah = 0; regs.h.al = 3; int86( 0x10, &regs, &regs );
        setpage(0); /* set display page to 0 */
        printf("This is page number 0\n"); getchar();
        setpage(1); /* set display page to 1 */
        printf("This is page number 1\n");
        /* Now write direct to screen 0 */
        x = 0; y = 1; attr = 0x82; /* column 0, row 1, green blinking */
        offset = (( y * 0x50 + x ) * 2 ) + ( 0 * 0x1000 );
        while ( *message ) {
                scrn[offset] = *message;
                ++offset;
                scrn[offset] = attr;
                ++offset;
                ++message;
        }
        getchar(); setpage(0); /* set display page to 0 */
}
```

There is a problem in writing text directly to the CGA screens. This causes **flicker** (snow). It is possible to incorporate a test which eliminates flicker. This involves only writing text to the screen during a horizontal retrace period.

The following program demonstrates the technique of direct video access, but waiting for the horizontal retrace to occur before writing a character.

```
main() {
        char far *scrn = (char far *) 0xb800000;
        register char attr = 04; /* red */
        register char byte = 'A';
        int loop, scrsize = 80 * 25 * 2;
        for( loop = 0; loop < scrsize; loop+= 2) {
                while( (inp(0x3da) & 01) == 0) ;
                while( (inp(0x3da) & 01) != 0) ;
                *scrn[loop] = byte;
                *scrn[loop+1] = attr;
        }
}
```

Medium Resolution Graphics mode (320.200)
Each pixel on the screen corresponds to two bits in the memory display buffer, which has values ranging from 0 to 3. Each scan line consists of eighty bytes, each byte specifying four pixels. All even row lines are stored in the even display bank, all odd row lines in the odd display bank (0x2000 apart). The display buffer begins at B800:0000, and the address of a particular row, column is found by,

```
offset = ((row & 1) * 0x2000) + (row / 2) * 0x50) + (column / 4)
```

Once the correct byte is located, the bits must be found. It is easiest to use a lookup table for this. In graphics modes there is no snow produced when directly updating the screen contents. The following portions of code illustrate some aspects of directly handling the video display.

```
bitset( row, column, color )
int row, column, color;
{
        int bitpos, mask[] = { 0x3f, 0xcf, 0xf3, 0xfc };
        char far *scrn = (char far *) 0xB8000000;
        unsigned int offset;
        color = color & 3;
        offset = ((row & 1) * 0x2000)+((row / 2) * 0x50) + (column / 4);
        bitpos = column & 3;
        color = color << 6;
        while( bitpos ) {
                color = color >> 2; bitpos;
        }
        scrn[offset] = scrn[offset] & mask[column & 3]; /* set bits off *
        scrn[offset] = scrn[offset] | color; /* set bits on/off */
}
```

Medium Resolution Graphics Color Modes
The ROM BIOS call int 10h allows the programmer to specify the color selections. The table below details how to use this call,

```
regs.h.ah = 0x0B;
regs.h.bh = palette;
regs.h.bl = color;
```

If register bh contains 0, bl contains the background and border colors. If register bh contains 1, bl

contains the palette being selected,

Palette	Color_Value	Color
0	0	Same as background
0	1	Green
0	2	Red
0	3	Brown
1	0	Same as background
1	1	Cyan
1	2	Magenta
1	3	White

High Resolution Graphics mode (640.200)
Each pixel on the screen corresponds to a single bit in the video display buffer. Two colors are supported, on or off (1 or 0). The home position is 0,0 and bottom right is 199,639 The display is split up into two areas, called odd and even. Even lines are stored in the even area, odd lines are stored in the odd area. One horizontal line contains eighty bytes, each byte represents eight pixels. To find the byte address for a particular co-ordinate, the formula is

```
offset = (( row & 1) * 0x2000 ) + ( row/w * 0x50 ) + ( column/8)
```

Having determined the offset byte (B800:offset), the following formula derives the bit position for the required pixel,

```
bit number = 7 - ( column % 8 )
```

To determine the status of the particular pixel (ie, set or off) requires the use of a bit mask. The following portions of code demonstrate this,

```
bittest( row, column) /* Title bittest(), return 1 if pixel set, else ret
int row, column;
{
        static int mask[] = {0x80,0x40,0x20,0x10,0x08,0x04,0x02,0x01};
        int byte, bitpos;
        char far *scrn = (char far *) 0xB8000000;
        unsigned int offset;
        offset = ((row & 1) * 0x2000)+((row / 2) * 0x50) + (column / 8);
        byte = scrn[offset]; bitpos = 7 - ( column % 8 );
        return( ((byte & mask[bitpos]) > 0 ? 1 : 0) );
}

bitset( row, column, color ) /* TITLE bitset(), set bit at row, column to
int row, column, color;
{
        static int mask[] = {0xfe,0xfd,0xfb,0xf7,0xef,0xdf,0xbf,0x7f};
        int bitpos;
        char far *scrn = (char far *) 0xB8000000;
        unsigned int offset;
        color = color & 1;
        offset = ((row & 1) * 0x2000)+((row / 2) * 0x50) + (column / 8);
        bitpos = 7 - ( column % 8 );
        color = color << bitpos;
        scrn[offset] = scrn[offset] & mask[bitpos]; /* set bit off first
        scrn[offset] = scrn[offset] | color; /* set bit on or off */
}
```

GRAPHIC ROUTINES

Creating Lines

Straight horizontal or vertical lines are relatively simple. However, diagonal lines are relatively complex to draw, as decisions need to be made as to where the dots must go, not where they should go according to a formula. General formula used in line calculations are,

```
line start points x1, y1
line end points x2, y2
```

Then the slope of the line (M) = (y2 - y1) / (x2 - x1)

and the Y intercept (B) = y1 - M * x1

An algorithm for plotting lines which is well known in computer circles is Bresenham's algorithm. I refer you to the book "Assembly language primer for the IBM-PC & XT: R Lafore, page 334". A diagonal line routine is present in the library.

Circles

Circles are easier to generate than lines. The basic equations involved in specifying a circle are, Given an angle in radians called A, and the centre of the circle as XC, YC and a radius of R, then

```
X = XC + R * COS(A)
Y = YC + R * SIN(A)
```

where X and Y are the two circle co-ordinates to plot next. Angles are always specified in radians. The circle function is present in the library.

Resolution Factors Since the video screen is not a 1:1 relationship, the x or y factor factor needs scaling by a specifed amount. The amount to scale by is depicted below.

Medium Resolution correction is 200/320 = .92

High Resolution correction is 200/640 = .46

WINDOWS

Windows are small viewing areas located on the display screen. Each application is run in a seperate window, and thus allows the user to view many events simultaneously. The following program illustrates the setting up of a window using the C language. Typically, the area where the new window is created would need to be saved for later recall, this can be done using the techniques illustrated under the section Memory accessing. A structure which contains an array of windows or pointers to windows can be used to implement multiple windows.

```
/* WINDOWS.C A window demonstration program */
/* Enter a backslash to quit program */
#include <dos.h>
#include <conio.h>
#define ENTER_KEY 0xd
#define BACK_SLASH '\\'

clear_window( tlr, tlc, brr, brc, attr )
unsigned int tlr, tlc, brr, brc, attr;
```

```
{
        union REGS regs;
        regs.h.ah = 6; regs.h.al = 0; regs.h.bh = attr;
        regs.h.ch = tlr; regs.h.cl = tlc; regs.h.dh = brr;
        regs.h.dl = brc; int86( 0x10, &regs, &regs );
}

setcursor( row, column )
int row, column;
{
        union REGS regs;
        regs.h.ah = 15; int86( 0x10, &regs, &regs ); /* get active page *
        regs.h.ah = 2; regs.h.dh = row;
        regs.h.dl = column; int86( 0x10, &regs, &regs );
}

scrollup( tlr, tlc, brr, brc, attr )
unsigned int tlr, tlc, brr, brc, attr;
{
        union REGS regs;
        regs.h.ah = 6; regs.h.al = 1; regs.h.bh = attr;
        regs.h.ch = tlr; regs.h.cl = tlc; regs.h.dh = brr;
        regs.h.dl = brc; int86( 0x10, &regs, &regs );
}

main()
{
        int attr, tlc, tlr, brc, brr, column, row;
        union REGS regs;
        char ch;
        printf("Co-ordinates top-left (row,column) ");
        scanf ("%d,%d", &tlr, &tlc);
        printf("Co-ordinates bottom-right (row,column) ");
        scanf ("%d,%d", &brr, &brc);
        printf("Enter attribute for window ");
        scanf ("%d", &attr);
        clear_window( tlr, tlc, brr, brc, attr );
        column = tlc; row = tlr;
        loop:
                setcursor( row, column);
                ch = getche();
                if( ch == ENTER_KEY) {
                        ++row; column = tlc;
                }
                else if( ch == BACK_SLASH) {
                        exit(0);
                }
                else
                        ++column;
                if ( column > brc ) {
                        column = tlc; ++row;
                }
                if( row > brr ) {
                        scrollup( tlr, tlc, brr, brc, attr );
                        row = brr;
        }
        goto loop;
}
```

A FASTER PSET ROUTINE

The problem with the wdot() and pset() routines so far is that they are slow! For each pixel set, an interrupt must be generated, parameters passed and recieved. The overhead is therefore quite high, and a faster way must be found. A program is thus needed which translates the pixel row/column numbers into an absolute address which can be accessed via a FAR type pointer as outlined in the previous section Accessing Memory. The next code segment illustrates such a technique.

```
#include <dos.h> /* TITLE: CPSET.C */
char far *screen = (char far *) 0xb8000000; /* Video RAM screen */
char far *parm = (char far *) 0x00400000; /* ROM BIOS DATA SEG */

cpset( x, y, color ) /* A fast PSET() routine! */
int x, y, color;
{
        /* defaults are for 640x200 graphics mode */
        int shift=3, max=7, mask=0x7f, rotate=1, temp=1, bank, mode=0xC0;
        if( parm[0x49] == 4 ) /* 320.200 Color mode */
        {
                shift=2; max = 3; mask = 0x3f; rotate = 2; temp = 3, mode
        }
        bank = y & 1; /* get bit zero */
        y = ( y & 0xfe ) * 40; /* adjust address to point to line*/
        if( bank ) /* select odd or even bank */
                y += 0x2000;
        y += ( x >> shift ); /* add columns to address */
        x = x & max; /* select column bits for position in byte */
        color = color & temp; /* valid color ranges only */
        if( parm[0x49] == 4 )
                color = color << 6; /* shift color bits into place */
        else
                color = color << 7; /* this for 640.200 BW mode */
        while( x ) {
                mask = ( mask >> rotate ) + mode; /* rotate bits to requi
                color = color >> rotate; /* position in both mask & color
                x++;
        }
        screen[y] = screen[y] & mask; /* erase previous color */
        screen[y] = screen[y] | color; /* insert new color */
}
```

Dedicated routines, ie, seperate functions for medium and high res as shown earlier give significant increases in speed at the expense of hardware dependance.

ASSEMBLY LANGUAGE INTERFACING

C programs may call assembly language routines (or vsvs). These notes concentrate on interfacing a machine code program which generates sound for a C program.

Assembly language routines which are called from C must preserve the 8088/86 BP, SI and DI registers. The recommended sequence for saving and restoring registers is,

```
entry: push bp
mov bp, sp
push di
push si
exit: pop si
pop di
mov sp, bp
```

```
pop bp
ret
```

The assembly language routine should be declared as **extern**al inside the C program. In writing the asm routine, it should be declared as public and the routine name should be preceeded by an underscore '_' character. The C program, however, does not use the underscore when calling the asm routine. The asm routine should also be declared as a PROC FAR type, in order for the linker to function correctly. The memory model used must be large, else the reference to FAR procedures will generate an error.

The following code illustrates how this is all done by way of a practical example.

```
;ASOUND.ASM Makes machine gun sound, firing a number of shots
;This is a stand-alone assembly language program!
;*****************************************************
stck segment stack ;define stack segment
db 20 dup ('stack ')
stck ends
;*****************************************************
code segment ;define code segment
main proc far ;main part of program
assume cs:code,ds:code
;set up stack for return to DOS
start: push ds ;save old data segment
sub ax,ax ;put zero in AX
push ax ;save it on stack
mov cx,20d ;set number of shots
new_shot:
push cx ;save count
call shoot ;sound of shot
mov cx,400h ;set up silent delay
silent:loop silent ;silent delay
pop cx ;get shots count back
loop new_shot ;loop till all shots done
ret ;return to DOS
main endp ;end of main part of program
;
;SUBROUTINE to make brief noise
shoot proc near
mov dx,140h ;initial value of wait
mov bx,20h ;set count
in al,61h ;get port 61h
and al,11111100b ;AND off bits 0 and 1
sound: xor al,2 ;toggle bit 1 into al
out 61h,al ;output to port 61
add dx,9248h ;add random bit pattern
mov cl,3 ;set to rotate 3 bits
ror dx,cl ;rotate it
mov cx,dx ;put in CX
and cx,1ffh ;mask off upper 7 bits
or cx,10 ;ensure not too short
wait: loop wait ;wait
dec bx ;done enough
jnz sound ;jump if not yet done
;turn off sound
and al,11111100b ;AND off bits 0 and 1
out 61h,al ;turn off bits 0 and 1
ret ;return from subroutine
shoot endp
;
code ends ;end of code segment
```

```
end start ;end assembly
```

The above program, called **ASOUND.ASM**, is a stand-alone machine code program. In order to interface this to a C program as a function which accepts the number of shots fired, the following changes are made.

```
;SOUND.ASMMakes machine gun sound, firing a number of shots
NAME SOUND
PUBLIC _bang
;
stck segment stack
db 200 dup ('stack ')
stck ends
;
BUZZ segment byte PUBLIC 'CODE' ;segment name
assume cs:BUZZ,ds:BUZZ
_bang PROC FAR ;main part of program
push bp ;save current bp
mov bp,sp ;get stack pointer
push di ;save register variables from c
push si
mov cx,[bp+6] ;get passed int value into CX
new_shot:
push cx ;save count
call shoot ;sound of shot
mov cx,400h ;set up silent delay
silent:loop silent ;silent delay
pop cx ;get shots count back
loop new_shot ;loop till all shots done
;return to DOS
pop si ;restore register varaibles
pop di ;restore register variables
mov sp,bp ;recover stack pointer
pop bp
ret ;back to C program
_bang endP ;end of main part of program
;
;SUBROUTINE to make brief noise
shoot PROC NEAR
mov dx,140h ;initial value of wait
mov bx,20h ;set count
in al,61h ;get port 61h
and al,11111100b ;AND off bits 0 and 1
sound: xor al,2 ;toggle bit 1 into al
out 61h,al ;output to port 61
add dx,9248h ;add random bit pattern
mov cl,3 ;set to rotate 3 bits
ror dx,cl ;rotate it
mov cx,dx ;put in CX
and cx,1ffh ;mask off upper 7 bits
or cx,10 ;ensure not too short
wait: loop wait ;wait
dec bx ;done enough
jnz sound ;jump if not yet done
;turn off sound
and al,11111100b ;AND off bits 0 and 1
out 61h,al ;turn off bits 0 and 1
ret ;return from subroutine
shoot endP
BUZZ ends
END
```

The above shows a listing for **SOUND.ASM**, the converted routine from ASOUND.ASM, which will interface to a C program. Note that **_bang()** has been declared as PUBLIC and a procedure of type FAR. It is assembled using MASM V4.00 using the following command line,

```
A>MASM /MX SOUND;
```

This creates a file called **SOUND.OBJ** which will be linked with the C object code shortly. The purpose of the switch /MX is to preserve case sensitivity for public names.

The C Compiler uses the DI and SI registers for variables declared register based, so these are saved by the asm routine. If the direction flag is altered by the machine code program then the instruction cld should be executed before returning. C programs pass parameters on the stack. These are accessed as follows,

```
                                NEAR CALL       FAR CALL
1st argument Single Word        [ bp + 4 ]      [ bp + 6 ]
Next arg, Single Word           [ bp + 6 ]      [ bp + 8 ]
Double Word                     [ bp + 8 ]      [ bp + 10 ]
```

Single words occupy two bytes, double words four bytes.

The C program which calls the assembly language routine _bang() is,

```
extern far bang(); /* CSOUND.C */
main() {
        int fires = 0, sam = 0;
        printf("Enter in the number of times you wish to fire the gun:\n"
        scanf("%d", &fires);
        for( ; sam < fires; sam++ ) {
                printf("Entering bang to fire the gun %d times.\n", sam);
                bang( fires );
        }
}
```

At linking time, the file sound.obj is added to csound.obj, ie,

```
; for Microsoft C Compiler
; LINK csound.obj+sound.obj
; for TurboC Compiler
; tlink c0l csound sound, csound, csound, cl
```

NOTE: The assembly language routine _bang() is declared as an external function of type FAR. When referenced in the C program, the asm routine _bang() is done so without the leading underscore character.

RETURN VALUES FROM ASM ROUTINES

```
Return Data Type                Register(s) used
Characters                        AX
Short, Unsigned                   AX
Integers                          AX
Long Integers                     DX = High, AX = Low
Unsigned Long                     DX = High, AX = Low
Structure or Union                Address in AX
```

```
Float or double type            Address in AX
Near Pointer                    Offset in AX
Far Pointer                     Segment = DX, Offset = AX
```

The following section deals with how parameters are passed between C functions at the machine code level.

PARAMETER PASSING AND ASSEMBLY LANGUAGE

ACCESSING THE STACK FRAME INSIDE A MODULE Lets look at how a module handles the stack frame. Because each module will use the BP register to access any parameters, its first chore is to save the contents of BP.

```
push bp
```

It will then transfer the address of SP into BP, so that BP now points to the top of the stack.

```
mov bp,sp
```

thus the first two instructions in a module will be the combination,

```
push bp
mov bp,sp
```

ALLOCATION OF LOCAL STORAGE INSIDE A MODULE Local variables are allocated onto the stack using a

```
sub sp, n
```

instruction. This decrements the stack pointer by the number of bytes specified by n. For example, a C program might contain the declaration

```
auto int i;
```

as defining a local variable called **i**. Variables of type auto are created on the stack, and assuming an integer occupies two bytes, then the above declaration equates to the machine code instruction

```
sub sp, 2
```

Pictorially, the stack frame looks like,

```
|----------|
|  ihigh   | < SP
|----------|
|  ilow    |
|----------|
|  BPhigh  | < BP
|----------|
```

```
|  BPlow   |
|----------|
```

The local variable i can be accessed using SS:BP - 2, so the C statement,

```
i = 24; is equivalent to mov [bp - 2], 18
```

Note that twenty-four decimal is eighteen hexadecimal.

DEALLOCATION OF LOCAL VARIABLES WHEN THE MODULE TERMINATES When the module terminates, it must deallocate the space it allocated for the variable i on the stack. Referring to the above diagram, it can be seen that BP still holds the top of the stack as it was when the module was first entered. BP has been used for two purposes,

```
- to access parameters relative to it
- to remember where SP was upon entry to the module
```

The deallocation of any local variables (in our case the variable i) will occur with the following code sequence,

```
mov sp, bp ;this recovers SP, deallocating i
pop bp ;SP now is the same as on entry to module
```

THE PASSING OF PARAMETERS TO A MODULE Consider the following function call in a C program.

```
add_two( 10, 20 );
```

C pushes parameters (the values 10 and 20) right to left, thus the sequence of statements which implement this are,

```
push ax ;assume ax contains 2nd parameter, ie, integer value 20
push cx ;assume cx contains 1st parameter, ie, integer value 10
call add_two
```

The stack frame now looks like,

```
|----------|
|  return  | < SP
|----------|
|  address |
|----------|
|    0A    | 1st parameter; integer value 10
|----------|
|    00    |
|----------|
|    14    | 2nd parameter; integer value 20
|----------|
|    00    |
|----------|
```

Remembering that the first two statements of module add_two() are,

```
add_two: push bp
mov bp, sp
```

The stack frame now looks like (after those first two instructions inside add_two)

```
|----------|
|  BP high | <- BP <- SP
|----------|
|  BP low  |
|----------|
|  return  |
|----------|
|  address |
|----------|
|    0A    | 1st parameter; integer value 10
|----------|
|    00    |
|----------|
|    14    | 2nd parameter; integer value 20
|----------|
|    00    |
|----------|
```

ACCESSING OF PASSED PARAMETERS WITHIN THE CALLED MODULE It should be clear that the passed parameters to module add_two() are accessed relative to BP, with the 1st parameter residing at [BP + 4], and the 2nd parameter residing at [BP + 6].

DEALLOCATION OF PASSED PARAMETERS The two parameters passed in the call to module add_two() were pushed onto the stack frame before the module was called. Upon return from the module, they are still on the stack frame, so now they must be deallocated. The instruction which does this is, add sp, 4 where SP is adjusted upwards four bytes (ie, past the two integers).

EXAMPLE C PROGRAM AND CORRESPONDING ASSEMBLER CODE Consider the following C program and equivalent machine code instructions.

```
add_two: push bp
mov bp, sp
add sp, 2
mov ax, [bp + 4]
add ax, [bp + 6]
mov [bp - 2], ax
mov sp, bp
pop bp
ret
main: push bp
mov bp, sp
sub sp, 4
mov [bp - 2], 0A
mov [bp - 4], 14
push wptr [bp - 4]
push wptr [bp - 2]
call add_two
add sp, 4
mov sp, bp
pop bp
ret
```

```
add_two( numb1, numb2 )
int numb1, numb2;
{       auto int result;
        result = numb1 + numb2;


}

main()
{
        int num1, num2;
        num1 = 10;
        num2 = 20;
        add_two( 10, 20 );

}
```

KEYBOARD BIOS CALLS

INT 16H This interrupt in the ROM BIOS provides for minimal character transfer from the keyboard. It is entered by first specifying the desired task to perform in the AH register.

```
AH = 0 Get Key
        Returns with AH = scan code
        AL = ascii char, 0 = non-ascii, ie Func key
AH = 1 Get status
        Returns with zflag = 0, valid key in queue
        = 1, no key in queue
AH = scan code
        AL = ascii char, 0 = non-ascii, ie, Func key
AH = 2 Get shift status
        Returns with AL = 7 Right shift 1 = pressed
        6 Left shift
        5 Ctrl
        4 Alt
        3 Scroll Lock 1 = on
        2 Num Lock
        1 Caps Lock
        0 Ins
```

Lets develop routines similar to those found in some libraries.

```
#include <dos.h>

int bioskey( cmd )
int cmd;
{
        union REGS regs;
        regs.h.ah = cmd;
        int86( 0x14, &regs, &regs );
        return( regs.x.ax );
}

int kbhit()
{
        union REGS regs;
        regs.h.ah = 1;
        int86( 0x16, &regs, &regs );
        return( regs.x.cflags & 0x0040 ); /* return Z flag only */
}
```

EQUIPMENT BIOS CALLS

INT 11H This interrupt is used to determine the hardware attached to the computer system. It returns a value in register AX, which is comprised as follows,

```
Bits     Description
15,14    Number of printers
13       Not used
12       Game I/O attached
11,10,9  Number of RS232 cards attached
```

```
8         Not used
7,6       Number of disk drives 00=1, 01=2, 10=3, 11=4
5,4       Initial video mode 00=40, 01=80, 11=Mono
3,2       Ram Size
1         Co-Processor
0         IPL from disk 1=disk, 0=None
Bits 0 - 7 correspond to SW1 settings on motherboard (port 60h)
```

Lets demonstrate one use of this. First, lets create a similar function to that provided by the TurboC compiler.

```
#include <dos.h>
int biosequip()
{
        union REGS regs;
        int86( 0x11, &regs, &regs );
        return( regs.x.ax );
}
```

Now, using this, lets develop a function to test if the machine has a serial card connected.

```
int is_serial_card()
{
        if( (biosequip() & 0xE00) == 0 )
                return 0;
        else
                return 1; /* Serial card is present */
}
```

MEMORY SIZE BIOS CALLS

INT 12H This interrupt returns the number of 1kbyte memory blocks present in the system. This value is returned in the AX register. Lets develop routines similar to those in TurboC.

```
#include <dos.h>

int biosmemory()
{
        union REGS regs;
        int86( 0x12, &regs, &regs );
        return( regs.x.ax );
}
```

or, what about this version, utilising TurboC's register variables.

```
int biosmemory()
{
        geninterrupt( 0x12 );
        return _AX;
}
```

RS232 BIOS CALLS

INT 14H This interrupt provides support for RS232 communications. The AH register value, on entry, determines the function to be performed.

```
AH = 0 reset port, DX = 0 = com1
        return value in AL
                7,6,5 Baud rate 000 = 110 100 = 1200
                001 = 150 101 = 2400
                010 = 300 110 = 4800
                011 = 600 111 = 9600
                4,3 Parity (00,10=off) (01=odd, 11=even)
                2 Stops (0=One, 1=Two stops)
                1,0 Word Size (10=7,11=8)
AH = 1 xmit a character, character in AL
AH = 2 recieve a character, returns character in AL
AH = 3 return status
```

Now, lets develop routines in C to program the rs232 card using the int 14 BIOS call, ie, the bioscom () function in TurboC.

```
#include <dos.h>
int bioscom( cmd, byte, port )
int cmd;
char byte;
int port;
{
        union REGS regs;
        regs.x.dx = port;
        regs.h.ah = cmd;
        regs.h.al = byte;
        int86( 0x14, &regs, &regs );
        return( regs.x.ax );
}
```

Now, lets develop routines to initialise the specified comport, and to transmit and recieve characters, without resorting to using int 14h. These types of routines directly program the rs232 card, thus are ideal for embedded applications, ie, ROMMABLE code.

```
/*- Initiliase the RS232 serial card -*/
#define INP inportb
#define OUTP outportb
/* Defines for RS232 communications */
#define DLL 0 /* divisor latch low byte */
#define DLH 1 /* divisor latch high byte */
#define THR 0 /* transmit hold register */
#define RBR 0 /* recieve buffer register */
#define IER 1 /* interrupt enable register */
#define LCR 3 /* line control register */
#define MCR 4 /* modem control register */
#define LSR 5 /* line status register */
#define MSR 6 /* modem status register */
#define RTS 0x02 /* request to send */
#define CTS 0x10 /* clear to send */
#define DTR 0x01 /* data terminal ready */
#define DSR 0x20 /* data set ready */
#define RBF 0x01 /* bit 0 of LSR, rec buf full */
#define THRE 0x20 /* bit 5 of LSR, trans reg 0 */
#define DISINT 0x00 /* disable interrupts in IER */
```

```
#define ABRG 0x83 /* access baud rate generator */
/**/

void rs232_init( com_port, baud rate, parity, stops, word_size )
int com_port, baud_rate, word_size, stops;
char *parity;
{
        unsigned int divisorh, divisorl, format, acia[2];
        int far *bios = 0x004000001;
        acia[0] = *bios; /* pick up address of com1 routine */
        acia[1] = *(bios + 1); /* pick up address of com2 routine */
        OUTP(acia[com_port] + IER, DISINT ); /* disable ints */
        OUTP(acia[com_port] + LCR, ABRG ); /* access baud rate gen*/
        switch( baud_rate ) {
                /* rem case 75, 110, 135, 150, 200, 1800, 19200 */
                case 300 : divisorh = 01; divisorl = 0x80; break;
                case 600 : divisorh = 00; divisorl = 0xc0; break;
                case 1200 : divisorh = 00; divisorl = 0x60; break;
                case 2400 : divisorh = 00; divisorl = 0x30; break;
                case 4800 : divisorh = 00; divisorl = 0x18; break;
                case 9600 : divisorh = 00; divisorl = 0x0c; break;
                default: printf("\nrs232_init: Error: Baud rate invalid.\
                        return -1;
        } /* end of switch */
        OUTP(acia[com_port] + DLL, divisorl );
        OUTP(acia[com_port] + DLH, divisorh );
        format = 0; /* This sets bit 6 and 7 to zero */
        if( (strcmp( parity, "E" ) == 0) || (strcmp( parity, "O" ) == 0)
                format = format  0x28; /* set bit 3 and 5 */
                if( strcmp( parity, "E" ) == 0 )
                        format = format  0x10; /* set bit 4 */
        }
        if( stops == 2 )
                format = format  0x04;
        switch( word_size ) {
                case 8 : format = format  0x03; break;
                case 7 : format = format  0x02; break;
                case 6 : format = format  0x01; break;
                case 5 : break;
                default: printf("\nrs232_init: Unsupported word length.\n
                        return -1;
        } /* end of switch */
        OUTP(acia[com_port] + LCR, format );
        return 0;
}

/* Transmit a single character to RS232 card -*/
void transmit( byte )
char byte;
{
        OUTP(acia[comport] + MCR, (RTS | DTR) ); /* assert RTS and DTR */
        while((INP(acia[comport] + LSR) & THRE)==0) /* trans reg empty? *
                ;
        OUTP(acia[comport] + THR, byte ); /* write character to THR */
        OUTP(acia[comport] + MCR, 0 );
}

/* Receive a single character from RS232 card */
char receive()  {
        char byte;
        OUTP(acia[comport] + MSR, (RTS | DTR) );
        while((INP(acia[comport]+LSR)&RBF)==0) /* has Data arrived? */
                ;
        OUTP(acia[comport] + MCR,0); /* stop all data */
```

```
        byte = INP(acia[comport] + RBR); /* get byte RBR */
        return( byte );
}
```

PRINTER SERVICES

INT 17H This interrupt provides support for the parallel printer. The AH register value, on entry, determines the function to be performed.

```
AH = 0 Write Character
        AL = character
        DX = printer number (0-2)
        Returns with AH = status code
                Bit 7 = printer not busy
                6 = acknowledge
                5 = out of paper
                4 = printer selected
                3 = I/O error
                2,1 = unused
                0 = time-out
AH = 1 Initialise Printer
        DX = printer number (0-2)
        Returns with AH = status code
AH = 2 Get Printer Status
        DX = printer number (0-2)
        Returns with AH = status code
```

Now lets develop a few routines which illustrate this,

```
int biosprint( command, ch, printer )
int command;
char ch;
int printer;
{
        _AH = command;
        _DX = printer;
        _AL = ch;
        geninterrupt( 0x10 );
        _AX = _AX >> 8;
        return( _AX );
}
```

INDEX Previous Next